HOTELPRO

Hotel Booking & Management System

FULL-STACK COURSE PROJECT | MERN STACK

Purpose: build the application described by the supplied course documentation, preserve its hotel-specific scope, and verify the user flows before submitting GitHub + demo video links.

DOCUMENT STATUS

Planning and specification only. The current repository contained documentation and reference materials, not a working application source tree.

Working name: HotelPro (Roomly appears in one source conclusion; pending confirmation)

1. The solution

HotelPro is a MERN-stack hotel booking and management application. Visitors can view a public landing page and room highlights. Customers can register, log in, browse rooms, select arrival/departure dates, create bookings, review booking history and cancel eligible bookings. Administrators can view all bookings, view rooms, add rooms and view registered users.

The source documents specify React with Vite, Node.js/Express, MongoDB/Mongoose, Axios, JWT authentication, password hashing and an MVC-style backend. MDBTabs is specifically named for the tabbed admin interface.

2. Users and core functionality

Role	Required experience
Visitor	Landing page, Home/Login/Sign Up navigation, room highlights/carousel, About section and room preview.
Customer / ordinary user	Register/login, personalized dashboard, room details and prices, arrival/departure date selection, booking, My Bookings, status display, eligible cancellation and logout.
Administrator	Authorized login, All Bookings, All Rooms, Add Room and All Users tabs, personalized greeting and logout.

3. High-level user journey

1. Visitor opens Home and chooses Login or Sign Up.
2. Registration validates the documented fields and creates an account; login validates credentials and routes to the appropriate dashboard.
3. Customer browses available rooms, selects dates and submits a booking.
4. The backend validates dates and availability, calculates the total and persists the booking.
5. Customer reviews My Bookings and cancels an eligible booking.
6. Admin opens the tabbed dashboard to view bookings, view/add rooms and inspect users.

4. Proposed technical structure

The following structure is a proposed normalization because original setup documents differ on folder capitalization and entry-point names.

- client/ — React + Vite pages/components, routes and Axios services.
- server/ — Express startup, routes, controllers, middleware, configuration and Mongoose models.
- MongoDB — User, Room and Booking records.
- Shared repository root — README, environment examples, specification documents and reference assets.

5. Core data model

Entity	Source attributes / meaning
User	user_id, name, email, password, c_password, phone, type. Other screenshots use role-like/user details differently; see the conflict register.
Room	room_id, roomNumber, description, photos, type, rent, count, currentBookings. Clarify whether records represent physical rooms or room-type inventory.
Booking	booking_id, room_id, user_id, fromdate, todate, totaldays, status, totalAmount, name.
CardDetails	The original ER diagram lists cardNumber, cvvcode, expmonthyear and cardholdername. Do not implement raw-card/CVV persistence; use safe provider references or a simulated payment step until requirements are confirmed.

6. Important source inconsistencies

Topic	Source difference	Proposed default / action
Product name	HotelPro in overview/workspace; Roomly in one conclusion.	HotelPro is the proposed display name; owner can confirm.
Entry point	server.js vs index.js.	index.js is proposed because detailed backend/execution notes use it.
Backend port	Screenshot and execution guide appear to differ.	8001 is the proposed default from execution instructions; verify against actual code.
Database	DARK, Roomly and hotel appear in source examples.	hotel is proposed default for local setup; owner can confirm.
Role signup	Source asks for Admin/Ordinary selection; security requires not trusting public role input.	Keep customer/admin behavior, provision admin through trusted method, and confirm expected visible selector.
Payments	Overview says payment processing; ER diagram lists raw card fields.	Simulated payment for course demo unless a real provider is explicitly required; never store raw card/CVV.

7. MVP priorities

- Complete authentication and role-protected APIs.
- Persist rooms and date-aware bookings.
- Make booking history and cancellation correct.
- Deliver all four admin tabs backed by real API data.
- Match the supplied screenshots closely enough that the UI represents the documented project.
- Make setup reproducible and verify both customer and admin flows before the demo.

8. Acceptance and submission

- Frontend build and backend startup verified.
- Database connection and persistence verified.
- Customer registration/login/room booking/history/cancellation verified.
- Admin login/All Bookings/All Rooms/Add Room/All Users verified.
- Role protection and booking ownership verified.
- README commands tested from a clean checkout.
- The final GitHub repository contains source code, not just planning documents.
- A demo video has actually been recorded, uploaded, checked for accessibility and submitted in Skill Wallet alongside the GitHub URL.

9. Source of truth

Read PRD.md, DECISIONS_AND_CONFLICTS.md, ARCHITECTURE.md, DATABASE.md, API_SPEC.md, SECURITY.md, TESTING.md and DESIGN_SYSTEM.md before implementation. The conflict register preserves source differences rather than pretending the original notes were consistent. Do not claim any feature is implemented or verified until actual code and test evidence support that claim.